AI 코딩 실습 평가 문항

와인 시료의 성분 측정값으로 품종 분류하기

총 8문항, 100점, 시험 120분

식품 분석팀이 와인 시료의 성분을 측정했습니다. 13개 수치 측정값으로 각 시료가 세 포도 품종 중 어느 품종에서 만들어졌는지 예측하려고 합니다. 품종의 실제 이름 대신 0, 1, 2라는 번호를 사용합니다. 이 번호는 크기나 품질의 순서가 아닙니다. 화학 지식은 필요하지 않습니다.

이 모의고사는 세 범주 중 하나를 고르는 분류를 다룹니다. 사진과 리뷰 텍스트는 사용하지 않습니다. 신경망은 시료마다 품종 0, 1, 2에 대한 점수 세 개를 출력합니다.

- 공통 준비 셀을 먼저 실행하고, 각 문항 안의 셀은 위에서 아래로 실행하세요. 각 문항에 필요한 자료를 따로 제공하므로 다른 문항의 답안은 필요하지 않습니다. 같은 문항의 (b), (c)는 앞 소문항의 결과를 사용할 수 있습니다.

- 실행만 하세요 셀은 제공 코드입니다. 작성하세요 셀의 `None`이나 답안 작성 영역을 필요한 코드로 바꾸세요. 중간 변수를 추가하거나 여러 줄로 작성해도 됩니다. 지정된 변수 이름과 함수의 입력, 반환 형식은 유지하세요.

- 특정 정확도에 도달하는 것이 정답 조건은 아닙니다. 요구한 데이터 처리, 학습과 평가 과정을 수행하세요.

- 공통 참고자료를 사용할 수 있습니다. 인터넷, 외부 서비스와 생성형 AI는 사용하지 않습니다. 필요한 데이터와 준비 자료는 모두 포함되어 있습니다.

- 제출물은 답안과 실행 결과가 남아 있는 Jupyter Notebook 파일 하나입니다.

데이터와 문항 구성

OpenML Wine 데이터(ID 187)의 UCI 원천 자료를 사용합니다. 실제 측정 시료 178개와 수치 입력 13개입니다. 배포 파일은 scikit-learn에 포함된 동일 UCI 원천 자료로 준비했습니다.

자료 / 변수	의미
dev	정답이 포함된 개발용 데이터 142개입니다. 1번에서 학습용과 검증용으로 나눕니다.
new_samples	별도로 남긴 시료 36개입니다. 정답 label은 없으며, 8번에서 이 시료에 대한 기준 모델의 출력을 사용합니다.
sample_id	시료 식별자입니다. 모델의 예측 입력으로 사용하지 않습니다.
label	예측할 품종 번호입니다. 정수 0, 1, 2 중 하나입니다.
feature_cols	모델의 입력으로 사용할 13개 수치 열 이름의 목록입니다. 식별자와 정답은 제외되어 있습니다.

입력 열: `alcohol`, `malic_acid`, `ash`, `alcalinity_of_ash`, `magnesium`, `total_phenols`, `flavanoids`, `nonflavanoid_phenols`, `proanthocyanins`, `color_intensity`, `hue`, `od_ratio`, `proline`.

`alcohol`은 알코올 측정값, `color_intensity`는 색의 진하기입니다. 각 열의 화학적 정의를 외울 필요는 없습니다. 결측치는 없습니다. 필요한 문항에는 학습 자료의 평균과 표준편차로 표준화한 입력을 제공합니다. 검증 자료와 새 시료에도 같은 기준을 적용했습니다. 원본의 품종 번호 1, 2, 3은 0, 1, 2로 바꾸었습니다.

문항	내용	배점	앞 문항의 답안
1	품종 비율을 유지하며 데이터 나누기	12점	필요 없음
2	학습 자료의 분포 확인하기	10점	필요 없음
3	가까운 시료를 이용하는 분류기 학습하기	12점	필요 없음
4	전체 정확도와 품종별 오류 확인하기	14점	필요 없음
5	세 품종을 구분하는 신경망 구성하기	16점	필요 없음
6	반복 학습을 실행하고 손실 기록하기	18점	필요 없음
7	Dropout을 적용하고 평가 모드로 예측하기	10점	필요 없음
8	예측 확률을 이용해 재확인할 시료 찾기	8점	필요 없음

Notebook의 공통 준비 셀을 먼저 실행하세요. 필요한 라이브러리를 불러오고, `load_tables()`로 `dev`, `new_samples`, `feature_cols`를 준비합니다. 답안 작성과 실행은 Notebook에서 합니다.

1. 품종 비율을 유지하며 데이터 나누기 | 12점

dev는 공통 준비 코드에서 읽어 둔 pandas DataFrame입니다. 한 행이 시료 하나이며 정답 열은 `label`입니다. 각 품종의 개수를 확인하고, 전체의 약 75%를 학습용, 25%를 검증용으로 나누겠습니다. 학습용과 검증용에 각 품종이 비슷한 비율로 들어가도록 분할하세요.

(a) [4점] dev에서 품종별 시료 수를 세어 `counts`에 저장하세요. 품종 번호 0, 1, 2 순서로 정렬된 pandas Series여야 합니다.

(b) [8점] `train_test_split`으로 dev를 나누어 `train_df`, `valid_df`에 저장하세요. `test_size=0.25`, `random_state=23`을 사용하고, `stratify`에는 품종 정답을 전달하세요. 모든 열과 기존 행 인덱스는 유지하세요.

준비 코드 - 실행만 하세요

```
# 실행만 하세요
print('dev:', dev.shape)
```

답안 작성

```
# 작성하세요
# (a) 품종별 개수
counts = None

# (b) 학습용과 검증용 분할
train_df = None
valid_df = None
```

작성 후 Notebook의 확인 셀을 실행하세요. 코드 줄 수에는 제한이 없습니다.

2. 학습 자료의 분포 확인하기 | 10점

`stats_df`는 별도로 제공하는 학습 시료 106개의 원래 측정값입니다. 1번 답안이 없어도 사용할 수 있습니다. `alcohol`과 `color_intensity`를 살펴보고 품종별 측정값이 어떻게 분포하는지 확인하겠습니다.

(a) [5점] 품종별로 `alcohol`과 `color_intensity`의 평균을 계산하여 `group_mean`에 저장하세요. 행 인덱스는 품종 0, 1, 2 순서이며, 두 열은 위에 적힌 순서로 둡니다.

(b) [5점] 제공된 좌표축 `ax`에 산점도를 그리세요. 가로축 값은 `alcohol`, 세로축 값은 `color_intensity`이며, 점의 색은 `label`에 따라 달라야 합니다. 품종 번호를 수치 측정값에 합치거나 평균으로 바꾸지 마세요.

준비 코드 - 실행만 하세요

```
# 실행만 하세요
stats_df = ready_train_table()
print('통계를 낼 학습 시료 수:', len(stats_df))
```

답안 작성

```
# 작성하세요
# (a) 품종별 두 측정값의 평균
group_mean = None

# (b) 산점도
fig, ax = plt.subplots(figsize=(5, 3.5))
# 이곳에 산점도를 그리는 코드를 작성하세요.

ax.set_xlabel('Alcohol')
ax.set_ylabel('Color intensity')
ax.set_title('Training samples by class')
plt.show()
```

작성 후 Notebook의 확인 셀을 실행하세요. 코드 줄 수에는 제한이 없습니다.

3. 가까운 시료를 이용하는 분류기 학습하기 | 12점

k-최근접 이웃(kNN) 분류기는 입력이 가까운 학습 시료들의 품종을 참고하여 새 시료의 품종을 예측합니다. 여기서는 가까운 이웃 5개를 사용합니다. 거리 계산식은 직접 구현하지 않습니다.

준비 코드가 표준화된 `X_train`(106행, 13열), `X_valid`(36행, 13열)와 정답 `y_train`, `y_valid`를 만듭니다. 정답 배열에는 각 시료의 품종 번호가 들어 있습니다. 1번과 2번 답안은 필요하지 않습니다.

(a) [6점] `KNeighborsClassifier`로 이웃 수가 5인 모델 `knn`을 만들고 `X_train`, `y_train`으로 학습하세요.

(b) [6점] 학습한 모델로 `X_valid`의 품종을 예측하여 `pred`에 저장하세요. `pred`는 검증 시료마다 품종 번호 하나를 담는 길이 36의 배열입니다. 검증 정답으로 모델을 추가 학습하지 마세요.

준비 코드 - 실행만 하세요

```
# 실행만 하세요
d = ready_data()
X_train, X_valid = d['X_train'], d['X_valid']
y_train, y_valid = d['y_train'], d['y_valid']
print('학습 입력:', X_train.shape,
      '/ 검증 입력:', X_valid.shape)
```

답안 작성

```
# 작성하세요
# (a) 모델 생성과 학습
knn = None

# (b) 검증 시료의 품종 예측
pred = None
```

작성 후 Notebook의 확인 셀을 실행하세요. 코드 줄 수에는 제한이 없습니다.

4. 전체 정확도와 품종별 오류 확인하기 | 14점

한 분류기의 검증 결과를 따로 제공합니다. `actual`은 실제 품종, `guessed`는 예측 품종이며 같은 위치끼리 같은 시료입니다. 두 배열은 길이 36입니다. 3번 답안을 사용하지 않습니다.

정확도(accuracy)는 전체 시료 중 품종을 맞힌 비율입니다. 예를 들어 10개 중 8개를 맞히면 0.8입니다.

혼동행렬(confusion matrix)은 실제 품종과 예측 품종별 개수를 모은 표입니다. 이 문제에서는 행이 실제 품종, 열이 예측 품종이며 둘 다 0, 1, 2 순서입니다. `cm[2, 0]`은 실제 품종 2를 품종 0으로 잘못 예측한 개수입니다.

(a) [4점] `actual`과 `guessed`로 정확도를 계산하여 `acc`에 저장하세요. 백분율 80이 아니라 비율 0.8과 같은 형태로 저장합니다.

(b) [6점] `confusion_matrix`에 `labels=[0, 1, 2]`를 지정하여 3×3 혼동행렬 `cm`을 만드세요.

(c) [4점] `cm`을 이용하여 실제 품종 1인 시료 중 다른 품종으로 잘못 예측한 개수를 계산하고 `missed_1`에 저장하세요. 정답 품종 1을 맞힌 개수는 제외합니다.

준비 코드 - 실행만 하세요

```
# 실행만 하세요
actual, guessed = load_classification_example()
print('검증 시료 수:', len(actual))
```

답안 작성

```
# 작성하세요
# (a) 정확도
acc = None

# (b) 혼동행렬
cm = None

# (c) 품종 1의 오분류 개수
missed_1 = None
```

작성 후 Notebook의 확인 셀을 실행하세요. 코드 줄 수에는 제한이 없습니다.

5. 세 품종을 구분하는 신경망 구성하기 | 16점

측정값 13개를 받아 세 품종 각각에 대한 점수를 출력하는 작은 신경망을 만들겠습니다. 구조는 13개 입력 → Linear(출력 16개) → ReLU → Linear(출력 3개)입니다. 시료 B개를 넣으면 출력은 `(B, 3)`입니다.

세 점수는 아직 합이 1인 확률이 아닙니다. 아래에서 사용할 `CrossEntropyLoss`는 이 점수와 정수 정답을 바로 받습니다. 모델 마지막에 Softmax를 추가하지 마세요. 정답은 0, 1, 2 중 하나이며 원-핫 벡터로 바꾸지 않습니다.

(a) [8점] `nn.Sequential`을 사용해 위 구조의 신경망을 만들고 `net`에 저장하세요. 층의 순서와 입력, 출력 개수를 맞추세요.

(b) [4점] 다중 분류 손실 함수 `nn.CrossEntropyLoss`를 만들어 `loss_fn`에 저장하세요.

(c) [4점] `net`의 전체 가중치를 갱신하는 `torch.optim.Adam`을 만들고 `optimizer`에 저장하세요. 학습률 `lr`은 0.005입니다.

준비 코드 - 실행만 하세요

```
# 실행만 하세요
seed_all(23)
d = ready_data()
xb = torch.tensor(d['X_train'][:8], dtype=torch.float32)
yb = torch.tensor(d['y_train'][:8], dtype=torch.long)
print('입력 크기:', tuple(xb.shape),
      '/ 정답 크기:', tuple(yb.shape))
```

답안 작성

```
# 작성하세요
# (a) 분류 신경망
net = None

# (b) 손실 함수
loss_fn = None

# (c) 가중치 갱신 방법
optimizer = None
```

작성 후 Notebook의 확인 셀을 실행하세요. 코드 줄 수에는 제한이 없습니다.

6. 반복 학습을 실행하고 손실 기록하기 | 18점

5번과 독립적인 새 신경망 `train_net`을 제공합니다. 입력 13개, 은닉 출력 12개, 품종별 점수 3개의 작은 신경망입니다. `loader`는 학습 자료를 최대 16개씩 묶어서 전달합니다. 이 묶음이 배치입니다. 학습 자료 전체를 한 번 사용하는 것을 1 epoch라고 합니다.

준비 셀의 두 함수는 완성되어 있습니다. 기울기 초기화, 손실과 기울기 계산, 가중치 갱신도 이미 구현되어 있으므로 이 함수들은 수정하지 않습니다.

- `train_one_epoch(train_net, loader, train_loss_fn, train_opt)`: 1 epoch 학습하고 그동안의 평균 학습 손실을 실수로 반환합니다.

- `validation_loss(train_net, xv, yv, train_loss_fn)`: 현재 모델의 검증 손실을 실수로 반환합니다. 가중치는 바꾸지 않습니다.

`xv`, `yv`는 검증 입력과 정답입니다. 답안을 고쳐 학습을 다시 시작하려면 이 문항의 준비 셀부터 다시 실행하세요.

(a) [12점] `for` 반복문으로 1부터 80까지 총 80 epoch를 학습하세요. 매 epoch마다 먼저 학습 함수를 호출하고, 그다음 검증 손실을 구하세요. 두 손실을 `tr_loss`, `va_loss`에 저장한 뒤 `history`에 `epoch`, `train_loss`, `valid_loss` 세 키를 가진 딕셔너리를 하나씩 추가하세요. `epoch`에는 1부터 80까지의 번호가 들어갑니다.

(b) [6점] `history`를 pandas DataFrame `history_df`로 만드세요. 제공된 좌표축 `ax`에 가로축을 `epoch`로 하여 `train_loss`와 `valid_loss`를 두 선으로 그리세요. 각 선의 `label`은 각각 `train`, `valid`로 지정합니다.

준비 코드 - 실행만 하세요

Notebook의 이 문항 준비 셀에 데이터, 모델, 두 함수의 전체 코드가 있습니다. 그 셀을 먼저 실행한 뒤 아래 답안을 작성하세요.

답안 작성

```
# 작성하세요
# (a) 반복 학습과 기록
history = []
# 이곳에 반복문과 기록 코드를 작성하세요.

# (b) 학습 과정 그래프
history_df = None
fig, ax = plt.subplots(figsize=(5, 3.5))
# 이곳에 두 손실의 변화 그래프를 그리세요.

ax.set_xlabel('Epoch')
ax.set_ylabel('Cross-entropy loss')
if ax.lines:
    ax.legend()
plt.show()
```

작성 후 Notebook의 확인 셀을 실행하세요. 코드 줄 수에는 제한이 없습니다.

7. Dropout을 적용하고 평가 모드로 예측하기 | 10점

학습 자료에 지나치게 맞추는 현상을 줄이기 위해 Dropout을 적용해 보겠습니다. Dropout은 학습 중 일부 중간 출력값을 무작위로 0으로 바꾸는 방법입니다. 평가할 때는 이 동작을 끕니다. 이 문항에서는 성능 향상 여부를 비교하거나 추가 학습하지 않습니다.

새 모델은 13개 입력 → Linear(출력 24개) → ReLU → Dropout → Linear(출력 3개) 순서입니다. 5번과 6번 모델은 사용하지 않습니다. `sample_x`는 형태가 `(4, 13)`인 입력 텐서입니다.

(a) [6점] `nn.Sequential`로 위 구조의 모델을 만들어 `drop_net`에 저장하세요. `nn.Dropout`에서 0으로 바꿀 확률 `p`는 0.2입니다. Dropout은 ReLU 바로 뒤에 둡니다.

(b) [4점] 함수 `predict_scores(model, x)`를 완성하세요. 전달받은 모델을 평가 모드로 바꾸고, 기울기 계산 기록을 하지 않는 범위에서 `x`에 대한 점수를 구해 `out`에 저장하세요. 제공된 `return out`으로 반환합니다. 확률로 바꾸거나 가중치를 갱신하지 않습니다.

준비 코드 - 실행만 하세요

```
# 실행만 하세요
seed_all(31)
sample_x = torch.tensor(
    ready_data()['X_valid'][:4], dtype=torch.float32
)
```

답안 작성

```
# 작성하세요
# (a) Dropout이 있는 새 신경망
drop_net = None

# (b) 평가 전용 예측 함수
def predict_scores(model, x):
    out = None
    # 이곳에 평가 모드 설정과 예측 코드를 작성하세요.
    return out
```

작성 후 Notebook의 확인 셀을 실행하세요. 코드 줄 수에는 제한이 없습니다.

8. 예측 확률을 이용해 재확인할 시료 찾기 | 8점

학습용 자료로 학습한 기준 신경망이 별도 시료 36개에 대해 계산한 점수를 제공합니다. `scores`는 `(36, 3)` 크기의 PyTorch 텐서이며 열 0, 1, 2가 각 품종의 점수입니다. `ids`는 같은 순서의 시료 식별자를 담은 길이 36의 NumPy 배열입니다. 앞 문항의 모델이나 답안은 필요하지 않습니다.

점수를 Softmax로 변환하면 시료별 세 값의 합이 1인 예측 확률이 됩니다. 가장 큰 확률의 품종을 예측으로 선택하되, 가장 큰 확률이 0.95 미만이면 재확인 대상으로 표시하겠습니다. 정확히 0.95이면 재확인 대상이 아닙니다. 이 기준은 모의고사용 업무 규칙이며 실제 정답 확률이 보장된다는 뜻은 아닙니다.

(a) [4점] 각 시료의 세 점수에 Softmax를 적용하여 `probs`에 저장하고, 가장 큰 확률에 해당하는 품종 번호를 `labels`에 저장하세요. `probs`는 `(36, 3)`, `labels`는 `(36,)` 크기의 PyTorch 텐서여야 합니다. 동점이면 작은 품종 번호를 선택합니다.

(b) [4점] 재확인 대상의 시료 식별자만 `ids`에서 골라 `review_ids`에 저장하세요. `review_ids`는 입력 순서를 유지하는 1차원 NumPy 배열이어야 합니다. 대상이 없으면 빈 배열이어도 됩니다. 새로운 파일이나 결과표는 만들지 않습니다.

준비 코드 - 실행만 하세요

```
# 실행만 하세요
ids, saved_scores = load_reference_scores()
scores = torch.tensor(saved_scores, dtype=torch.float32)
print('점수 크기:', tuple(scores.shape))
```

답안 작성

```
# 작성하세요
# (a) 예측 확률과 품종
probs = None
labels = None

# (b) 재확인할 시료 식별자
review_ids = None
```

작성 후 Notebook의 확인 셀을 실행하세요. 코드 줄 수에는 제한이 없습니다.